

C64uRemote for the M5StickC Plus2

Technical Documentation

Version 1.0

Contents

Overview	2
Hardware	2
Target platform	2
Two separate I ² C buses	2
Detection at startup	3
Build environment	3
PlatformIO	3
Board settings — the dark-display trap	3
PSRAM	4
Memory budget	4
Configuration (build_env.h)	4
Wi-Fi subsystem	4
Runtime configuration instead of compile time	4
NVS layout	5
Card format	5
Setup portal	5
Direct mode	6
NFC subsystem	7
Polling strategy	7
Card types	7
Data format	7
Command cards	8
What the stick cannot do	8
Software architecture	8
State model	8
Navigation	8
Rendering	9
ReST interface to the C64 Ultimate	9
Refused connections	9
Own HTTP path (since v1.4.0)	10
NFC RF field	10
Reconnecting with several networks	10
Input logic	10
Buttons	10
Blocking calls	11
PowerOff safeguards	11
Persistence	11
Project layout	11

Troubleshooting	11
Sources	12
License	12
How this was made	12

Overview

This build of C64uRemote runs on an **M5StickC Plus2** and drives a **Commodore 64 Ultimate** through its ReST API. Three building blocks were added compared to the first StickC version, all ported from the M5Stack Core edition:

- **NFC through a Unit RFID2** on the Grove port — read Wi-Fi cards and command cards, and write both
- **Wi-Fi configuration at runtime** — up to four networks in NVS instead of fixed values from `build_env.h`
- **Setup portal** — a private access point with a small web interface

Everything that needs a microSD card was left out: file browser, upload to the C64, card dump and restore. The stick has no card slot.

The card format is deliberately identical to the M5Dial and M5Stack Core. A card created on any of the devices works on all of them.

Hardware

Target platform

SoC	ESP32-PICO-V3-02, 240 MHz, dual core
Flash	8 MB (in package)
PSRAM	2 MB (in package)
Display	ST7789, 135 × 240, used in landscape (240 × 135)
Backlight	GPIO 27, via PWM
Power hold pin	GPIO 4 — must be HIGH, or the device switches itself off on battery
Input	button A (GPIO 37), button B (GPIO 39), power button through the PMU

The ESP32-PICO-V3-02 matters here: it differs from the PICO-D4 of the old M5StickC by its **integrated PSRAM** and a different package marking in the eFuse (`pkg_ver == 6`). That marking is exactly how M5GFX recognises the Plus2 — see *Build environment*.

Two separate I²C buses

This is the central wiring decision of this build:

Bus	Pins	Device	Address
Wire	SDA = G0, SCL = G26	Hat MiniJoyC (HAT port)	0x54
Wire1	SDA = G32, SCL = G33	Unit RFID2 / WS1850S (Grove port)	0x28

Both run in parallel without getting in each other's way. The MFRC522 driver receives its bus instance in the constructor:

```
MFRC522_I2C rfid(kRfidAddr, -1, &Wire1);
```

Wire1.begin(32, 33, 100000UL) has to run before initRfid() — PCD_Init() does **not** start the bus itself.

The MiniJoyC is driven without a library: miniJoyBegin() starts the bus and pings the address, miniJoySetLedColor() writes three bytes to register 0x40, and reading goes through joyReadRegister(). That saves a dependency and the ambiguity warnings from that library's source.

Detection at startup

```
Wire.begin(0, 26, 100000)    -> ping MiniJoyC    -> miniJoyReady
Wire1.begin(32, 33, 100000) -> initRfid()    -> app.rfidReady
```

initRfid() first checks with an empty I²C transaction whether anything answers at 0x28, and only then calls PCD_Init(). The version register is merely logged and **not** evaluated: the WS1850S does not always report the same value there as a genuine MFRC522.

Build environment

PlatformIO

```
cp src/build_env.h.example src/build_env.h # fill in
pio run -t upload
pio device monitor
```

In VS Code it is enough to open the project folder; the PlatformIO IDE creates .vscode/c_cpp_properties.json itself. .vscode/extensions.json and .vscode/settings.json ship with the project and suggest the PlatformIO extension when you open the folder.

Board settings — the dark-display trap

There is **no dedicated board entry** for the StickC Plus2 (as of espressif32 6.9). One therefore uses the M5StickC entry — but not unchanged.

The board manifest m5stick-c.json sets -DARDUINO_M5Stick_C. M5GFX uses that marker in init_impl() as its starting point:

```
#elif defined ( ARDUINO_M5STICK_C ) || defined ( ARDUINO_M5Stick_C )
    nvs_board = board_t::board_M5StickC;           // = 3
```

and passes it to autodetect(). There, the branch for the PICO-V3-02 reads:

```
else if (pkg_ver == 6)           // PICOV3_02 (StickCPlus2 / ATOM PSRAM)
{
    if (board == 0 || board == board_t::board_M5StickCPlus2)
```

With the value 3 the condition is false. The branch is skipped, **no panel is initialised at all**, and the power hold pin G4 is never asserted. The result is a dark screen with no error message. The marker is not a hint, it is a commitment.

platformio.ini corrects this in three places:

```
board_build.extra_flags =                ; drop -DARDUINO_M5Stick_C

build_flags =
    -DM5GFX_BOARD=board_t::board_M5StickCPlus2
    -DBOARD_HAS_PSRAM

build_unflags =
    -DARDUINO_M5Stick_C
```

M5GFX_BOARD is checked ahead of the whole #elif chain and therefore wins. In addition setup() sets:

```
cfg.fallback_board = m5::board_t::board_M5StickCPlus2;
```

To verify: the serial monitor must show [Autodetect] M5StickCPlus2.

M5GFX stores a detected board in NVS and reads it back on the next start **before** any compile-time marker. If a wrong value ends up there, only `pio run -t erase` helps — which also wipes the stored Wi-Fi credentials.

PSRAM

The PICO-V3-02 has 2 MB of PSRAM in the package, but the Arduino core only calls `psramInit()` when `BOARD_HAS_PSRAM` is defined. Without that marker `ps_malloc()` simply returns `NULL`:

```
void *ps_malloc(size_t size){
    if(!spiramDetected){ return NULL; }
    ...
}
```

Hence `-DBOARD_HAS_PSRAM` in the `build_flags`. The code does not rely on it regardless:

```
uint16_t* allocFrameBuffer(size_t bytes) {
    uint16_t* buffer = nullptr;
    if (psramFound()) buffer = static_cast<uint16_t*>(ps_malloc(bytes));
    if (buffer == nullptr) buffer = static_cast<uint16_t*>(malloc(bytes));
    return buffer;
}
```

Memory budget

Buffer	Size	Location
M5Canvas (full-screen sprite)	$240 \times 135 \times 2 = 64.8 \text{ kB}$	internal
plainLogoPixels	64.8 kB	PSRAM, else internal
boxedLogoPixels	64.8 kB	PSRAM, else internal

While the **setup portal** is running, both logo caches are released: the access point and the web server need the internal heap. Three places belong together here and must be considered as a set when changing anything:

- `startPortal()` calls `releaseLogoCache()`
- `stopPortal()` calls `ensureLogoCache()`
- `render()` has a null-pointer branch for the home screen without a cache

If allocation fails, the device keeps running without logo effects and reports *WENIG SPEICHER* (low memory). An earlier revision looped forever at this point — a dark screen with no hint whatsoever.

Configuration (build_env.h)

```
#define C64U_WIFI_SSID      "MyNetwork"
#define C64U_WIFI_PASSWORD "MyWifiPassword"
#define C64U_TARGET_HOST   "192.168.0.64"
#define C64U_TARGET_PASSWORD ""
```

These values are **initial values only**. `loadNetConfig()` reads NVS first; only if nothing was ever stored there (the key `wcount` is missing) is the entry from `build_env.h` adopted as the first profile. The fields may therefore be left empty.

Wi-Fi subsystem

Runtime configuration instead of compile time

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax]; // kWifiProfileMax = 4
```

```

size_t      gWifiCount = 0;
size_t      gWifiTry   = 0;
String      gTargetHost, gTargetPass;

```

wifiAddProfile() moves a network to the front (most recently used first); when the list is full, the last entry drops out. beginWiFi() takes gWifiProfiles[gWifiTry] and then advances, so serviceWiFi() tries a different network every 10 s until one answers.

At startup setup() calls wifiPickBestProfile() once: a blocking scan that moves the known network with the strongest signal to the front. That only pays off with more than one stored network and is skipped otherwise.

NVS layout

Two namespaces, so that a *Factory Reset* of the display settings does not take the credentials with it:

c64uremote — settings

Key	Content
anim_on, fx_mode, anim_spd, fx_time, st_time, bright	display
joy_ovl, joy_led, joy_led_br, joy_thr	MiniJoyC
auto_nfc	background polling interval
card_cnf	prompt time of a PowerOff command card, in seconds

c64unet — network

Key	Content
host, hostpw	address and password of the c64u
wcount	number of stored networks (0-4)
s0...s3	SSIDs
p0...p3	passwords
dmode	direct mode on/off
dssid, dpass	SSID and password of the direct network
dnet	address range of the direct network, e.g. 192.168.4

wcount doubles as the marker “something has been saved here before”.

Card format

Wi-Fi cards use the scheme from the Wi-Fi QR code:

```
WIFI:S:MyNetwork;T:WPA;P:secret;;
```

parseWifiText() evaluates the fields S: and P:; T: is ignored (the type follows from the password). Special characters are escaped with a backslash; the parser works character by character with an escape state, so ; and : inside a password work. wifiCardText() is the counterpart for writing.

textLooksLikeWifi() only checks the prefix WIFI: — that is how background polling spots a Wi-Fi card without parsing it fully.

Setup portal

SSID	C64uRemote-Setup
Password	c64ultimate
Address	192.168.4.1
Timeout	5 min without use

Grace period after saving	2.5 s, so the response page still arrives
---------------------------	---

A `DNSServer` answers every name with its own address and `onNotFound` redirects to the start page — together that makes a captive portal which opens by itself on most devices.

Password fields left empty keep the previous value. If the SSID is already stored, its password is carried over.

Direct mode

For meetings without a router the Stick opens a WiFi network of its own (`WIFI_AP`, without a station part – for the same reason as the setup portal: a searching STA changes channel and throws connected devices off).

SSID / password	C64uRemote-Direct / c64ultimate (defaults, changeable)
Channel, max. clients	6, 8
Stick address	<net>.1
DHCP range	<net>.64 ... <net>.74
c64u target	<net>.64
<net>	192.168.4 (default) or 192.168.2, anything via <code>direct_net</code>

Start order: `WiFi.softAP()` first, then `WiFi.softAPConfig(ip, ip, 255.255.255.0, <net>.64)`. The fourth parameter sets the start of the DHCP range (Arduino core 2.0.17 hands out eleven addresses from there). The other way round, depending on the core version, the default 192.168.4.1 may stay – the setup portal never notices because it uses exactly that address.

Target address: in direct mode `targetHost()` returns `gDirectHost` instead of the stored home address, which stays untouched. If the device is connected as a normal client to a network whose SSID matches the direct network's, `<net>.64` applies as well – so a second remote can join without changing its settings.

Finding the c64u: the first DHCP lease is `.64`. If another device was quicker, `directNextCandidate()` keeps looking: candidates are `.64` and all addresses from `esp_wifi_ap_get_sta_list()` + `esp_netif_get_sta_list()`. After each failed status test the next one is tried. A device counts as the c64u if it answers `/v1/version` with JSON (or with 401/403 if a password is missing). Once an address has answered, it is only switched after the second failure in a row – the c64u occasionally refuses on its own (see *Refused connections*).

Status test: with nobody connected the test is skipped entirely. When a device joins (`WiFi.softAPgetStationNum()` rises) and the c64u is not confirmed yet, it is checked right away, starting again at `.64`. Since the c64u is often still busy with DHCP at that point, up to six follow-up checks follow at 3 s intervals (`directRecheckDue()`), on any page, until it answers. In the direct network the connection setup also has a 1 s deadline.

Resetting DHCP: the ESP32's DHCP server hands out addresses via a pointer that only moves forward. If the c64u releases its address when leaving or asks for the old one when coming back, the server drops the entry and hands out the next one (`.65`, `.66` ...). So `directRestartDhcp()` restarts the server as soon as the last device has left – afterwards it starts at `.64` again. With other devices still connected this is skipped, because the server forgets all assignments and could otherwise hand out an address twice; the search then finds the c64u at its new address.

Switching: `setDirectMode()` stores the setting, discards the connection state and restarts AP or WiFi. An explicitly chosen network (list, scan, WiFi card) ends direct mode. The setup portal displaces the direct AP temporarily; after the portal, `beginWiFi()` starts it again.

NVS (namespace `c64unet`): `dmode` (bool), `dssid`, `dpass`, `dnet`. Unusable values (SSID empty or > 32 characters, password < 8 or > 63, address range not three numbers 0...255) are replaced by the defaults on load.

NFC subsystem

Polling strategy

serviceRfid() has two modes:

On the card screens (NfcRead, NfcWrite, WifiCard) it asks every 250 ms with cardPresent() — full reader timeout.

On the home screen background polling runs at the *Auto-NFC* interval (1.5 s / 0.7 s / 0.3 s) and uses cardPresentQuick().

The difference matters for responsiveness. With no card present, the MFRC522 waits after the REQA command until its internal timer expires. PCD_Init() configures 0x03E8 = 1000 steps of 25 µs each — **25 ms** during which the main loop stalls. A card answers far sooner (frame delay time at 106 kbit/s is about 86 µs), so cardPresentQuick() lowers the timeout to 80 steps (2 ms) for the probe alone and restores the original value immediately afterwards — before card selection. Authentication, reading and writing therefore still run with the full timeout.

A card left on the reader is not executed over and over thanks to a 2.5 s lock (kRfidRepeatMs). The exception is the prompt of a PowerOff card: that may be confirmed by presenting the card again at any time.

Card types

Family	Access
NTAG213/215/216, MIFARE Ultralight	4 bytes per page, NDEF TLV from page 4, no key
MIFARE Classic 1K/4K/Mini	16 bytes per block, NDEF TLV from block 4, sector trailers skipped

For Classic cards authentication is attempted first with the NDEF key D3F7D3F7D3F7, then with the factory key FFFFFFFFFF. The last successful key is remembered per card, otherwise every block costs a pointless failed attempt plus a reselect.

Sector trailers and the MAD are never written. A card cannot be bricked by this program; an unformatted Classic card may afterwards only be readable on this device.

After a failed authentication the card sits in HALT state and only answers WUPA. reselect-Card() therefore wakes it deliberately with PICC_WakeupA() and selects it again — a plain PICC_IsNewCardPresent() (REQA) would no longer find it.

Data format

What gets written is always a single NDEF text record, UTF-8, language code en:

```
TLV      : 03 <len> ... FE
Record   : D1 01 <plen> 54 | 02 'e' 'n' | <text>
          D1 = MB|ME|SR|TNF=1 (Well Known), 54 = 'T'
```

The buffer is padded to a multiple of 16 so that whole blocks are written. After writing, write-CardText() reads the card back and compares the text.

When reading, the old raw format with the C64UPATH magic is also recognised, so previously written cards keep working.

The parser is deliberately tolerant: some writers store a wrong payload length — the TeensyROM writes a constant 0x10 there even though the TLV length is correct. For the last record (ME flag) the TLV length therefore takes precedence.

Command cards

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      switch off immediately
CMD:POWEROFF=8      ask first, 8 s to confirm
CMD:POWEROFF        ask first, using the device setting "NFC-Cmd PowOff"
CMD:M50FF           switch the Stick itself off (also CMD:DIALOFF)
CMD:CPU=10          set the CPU to 10 MHz
CMD:JOY             toggle the joystick ports (Normal <-> Swapped)
CMD:JOY=SWAPPED     set the ports fixed; also NORMAL, WASD1, WASD2
```

`parseCardCommand()` is insensitive to case and spaces. Without an argument, `POWEROFF` uses the device setting *NFC-Cmd PowOff* (3/5/8/15 s); 0 means “no prompt”.

The prompt is tracked in `app.cardPowerOffPending` together with the UID and an expiry time. It is confirmed by presenting **the same** card again or by a button press (`activateCurrent()` intercepts that ahead of everything else).

What the stick cannot do

`processCard()` recognises path cards but does not execute them:

```
app.rfidHint = "Programmkarte - dafuer fehlt die SD";
setModal("BRAUCHT SD-KARTE", ...);
```

Launching a program would require reading the file from a local microSD and streaming it to the C64. Both exist only in the Core and Dial editions.

Software architecture

State model

`ScreenMode` drives both rendering and input:

State	Content
Home	logo / effects, background polling of the reader
Menu	main menu
CpuMenu	clock speeds
Status	connection overview
DisplaySettings	settings list
NfcMenu	NFC / RFID submenu
NfcRead	present a card → execute its content
CmdPick	pick a command for a card
NfcWrite	present a card → write text
WifiMenu	Wi-Fi submenu
WifiCard	present a card → read credentials
WifiSaved	stored networks, optionally in delete mode
WifiPortal	setup access point running

Navigation

So that a new screen does not drag three more switch branches along with it, all input goes through three functions:

```
void moveSelection(int delta);      // scroll
void activateCurrent(uint32_t);    // select / execute
void goBack(uint32_t);             // one level back
```

listSelection() supplies the pointer and length of the current screen's list. Buttons and the MiniJoyC only call those three; the home screen's special cases (double click = reboot, joystick up = reset) live directly in the button handling.

activateCurrent() intercepts an open PowerOff prompt ahead of everything else.

Rendering

Drawing always goes into an M5Canvas and is pushed to the display in one piece — hence no flicker. The frame rate is about 30 fps (kFrameMs = 33).

On the home screen a redraw only happens when something changed (app.home.frameDirty) or an effect is running. The effects read from plainLogoPixels, the pre-rendered logo; boxedLogoPixels holds the same graphic with a cleared inner area.

ReST interface to the C64 Ultimate

The base is `http://<host>` with a 2.5 s timeout. If a password is configured it travels as the X-Password header.

Purpose	Method and path
Reachability / version	GET /v1/version
Reset	PUT /v1/machine:reset
Reboot	PUT /v1/machine:reboot
Power off	PUT /v1/machine:poweroff
Ultimate menu	PUT /v1/machine:menu_button
Configuration tree	GET /v1/configs
Read CPU value	GET /v1/configs/<category>/<item>
Set CPU value	PUT /v1/configs/<category>/<item>?value=<value>
Read/set joystick ports	the same /v1/configs paths

resolveCpuPath() looks the CPU setting up in the configuration tree once — the Ultimate firmware names it differently depending on the version. If that fails, a built-in fallback list of the usual clock speeds is used.

There is no machine: command for the joystick ports. They are a configuration item as well - in testing *Joystick Swapper* in *U64 Specific Settings*, with the values Normal, Swapped, WASD Port 2, WASD Port 1. resolveJoyPath() looks it up the same way (first *U64 Specific Settings*, otherwise every category until an item name contains "Joystick"). joyTokenFromValue() and joyValueFromToken() convert between the device value and the card token (WASD Port 1 <-> WASD1). One caveat when reading the source: everything else named joy belongs to the MiniJoyC on the HAT port.

refreshConnectionStatus() probes at most every 15 s, first without and then with the password. The four states of the status LED follow from that.

Refused connections

The HTTP server of the Ultimate firmware accepts only one connection at a time and refuses further ones with a TCP RST; HTTPClient reports this as *connection refused*. It was observed with only a single device on the network as well - so it happens sporadically and is neither a radio nor an address problem.

The actual call lives in sendApiRequestOnce(). sendApiRequest() is a wrapper around it: if no connection comes about (HTTPC_ERROR_CONNECTION_REFUSED), up to two more attempts follow after kApiRetryDelayMs (kApiConnectAttempts = 3). Retrying happens **only** in this case - nothing reached the c64u then, so a command cannot be doubled. Read errors and HTTP error statuses (4xx, 5xx) are passed through unchanged.

On top of that `refreshConnectionStatus()` saves a request: without a stored password the second query would be byte-identical to the first, because the X-Password header is only set when there is one. That halves the base load on the c64u.

Own HTTP path (since v1.4.0)

All requests to the c64u - `sendApiRequestOnce()` - no longer go through `HTTPClient/WiFiClient` but through small functions on lwIP sockets:

- `rawOpen()` connects without blocking and checks via `select()` without waiting every 2 ms; limit `kHttpConnectTimeoutMs` (1.5 s), in direct mode `kHttpConnectDirectMs` (1 s).
- `rawSendAll()` sends the request in pieces with `MSG_DONTWAIT`.
- `rawReadResponse()` reads the reply until the c64u closes the connection (or shortly after the last byte announced by Content-Length) and evaluates the status line, Content-Length and chunked. If the c64u closes without a reply the error is *connection lost*, if the reply does not come, *read Timeout*.

`rawHttpRequest()` combines this for GET/PUT; every request carries `Connection: close`. Background: with `HTTPClient` most requests got lost on the home network at times, while a connection checked without waiting got through reliably.

`staSsid()` and `staRssi()` return network name and signal strength from a cache that is refreshed at most once per second via `esp_wifi_sta_get_ap_info()`. `WiFi.SSID()` asks the driver on every call, and `targetHost()` needs the network name all the time (detecting the direct network as a client). `directStationCount()` asks for the number of stations in direct mode at most every 500 ms.

NFC RF field

The MFRC522 driver leaves the 13.56 MHz field on permanently after `PCD_Init()`. `rfidFieldOn()` and `rfidFieldOff()` switch it on only for the card probe (`cardPresent()`, `cardPresentQuick()`, then 5 ms for the card to power up) and while a detected card is being processed; on an empty probe it goes off again. After `processCard()`, however, it stays on until the card is gone (`rfidHoldCard()`, `rfidHeldCardGone()`): the processed card has been put to sleep with HLTA and no longer answers REQA; whether it is still there is checked with WUPA. Switching the field off and on would wake the card up fresh and it would be executed or written again. On the M5Dial, whose NFC antenna sits right next to the WiFi antenna, the permanently switched-on field disturbed WiFi reception; here switching it off mainly saves power.

Reconnecting with several networks

`beginWiFi()` moves on to the next stored profile after every attempt. Without a countermeasure that means: with two networks stored of which only one is reachable, every other reconnect after a dropout hits the dead one and costs a full retry cycle (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` therefore remembers the profile the connection came up on, via `wifiProfileIndex(staSsid())`, and makes it the next attempt; `gWifiNoted` keeps this to once per connection. After a dropout the first attempt goes back to the working network, and the dead one is only tried if the good one is really gone.

Input logic

Buttons

Button	Home screen	Lists
A	reset; twice within 300 ms = reboot	<code>goBack()</code>
B	Ultimate menu	<code>moveSelection(+1)</code>
Power (short)	open the menu	<code>activateCurrent()</code>

The single click on A is executed deliberately late: the reset only runs once 300 ms have passed without a second press. There is no other way to put reset and reboot on the same button.

Blocking calls

Two places deliberately stall the main loop:

- **HTTP requests** for up to 2.5 s
- **Waiting for the Wi-Fi connection** after a Wi-Fi card for up to 8 s (`kWifiCardConnectMs`), with the message *VERBINDE...* drawn beforehand

In both cases a short stall is the more honest option: the user should see the result, not a menu that stays operable while nothing is settled underneath.

PowerOff safeguards

Switching off is guarded in three places:

1. The menu entry and joystick-down always ask — a second press within 2 s (`kPowerOffConfirmMs`)
2. Command cards with a prompt: present again or press a button within 3–15 s
3. `CMD:POWEROFF=0` switches off without asking — whoever creates such a card means it

Persistence

What	Where
Display and input settings	NVS c64uremote
Wi-Fi profiles, host, host password	NVS c64unet
Detected board	NVS, managed by M5GFX itself

Factory Reset in the settings only affects c64uremote. Network data is cleared under *WLAN* → *Alle loeschen*, everything at once with `pio run -t erase`.

Project layout

```
M5StickC_Plus2_C64uRemote/
├── platformio.ini      board, libraries, partition, upload
├── README.md
├── LICENSE             MIT - Karl Prosser, Martin Oswald
├── .vscode/           recommended extensions, editor settings
├── docs-src/          markdown sources of the manuals
├── doc/               finished manuals (PDF)
├── src/              German edition
│   ├── main.cpp       entire firmware
│   ├── build_env.h    credentials (do not version)
│   ├── build_env.h.example  template
│   └── 1MHz_logo_rgb565.h  logo as an RGB565 array
├── src-en/
│   └── main.cpp       English edition, identical code
```

Dependencies: M5Unified, M5GFX, ArduinoJson and the I²C driver `kkloesener/MFRC522_I2C`. There is deliberately no library for the MiniJoyC — the four accesses that are needed sit directly in the source.

Troubleshooting

The screen stays dark. Check `platformio.ini` against the *Board settings* chapter. The serial monitor must show `[Autodetect] M5StickCPlus2`. If that does not help: `pio run -t erase`, then flash again.

Message *WENIG SPEICHER*. The logo caches did not fit. Check that `-DBOARD_HAS_PSRAM` is set; otherwise the monitor shows `logo cache alloc failed` along with the free heap.

Everything under *NFC / RFID* says *kein NFC*. The reader was not found at startup. Check the Grove plug; the monitor shows `RFID2 nicht gefunden`. The reader is only looked for once at startup, so restart after plugging it in.

Cards are read but not written. On MIFARE Classic the data blocks may be secured with a foreign key. This program only tries the NDEF and the factory key; a dictionary attack lives deliberately only in the Core edition.

The setup portal does not open. Some phones drop a Wi-Fi network without internet access on their own. Choose “stay connected anyway” in the Wi-Fi settings and enter `192.168.4.1` in the browser by hand.

Sources

- Original project: Karl Prosser (@klumtsy), <https://github.com/ReadyOS-C64/C64uRemote>
- Ultimate firmware and ReST API: <https://1541u-documentation.readthedocs.io>
- M5Unified / M5GFX: <https://github.com/m5stack/M5Unified>
- MFRC522 I²C driver: https://github.com/kkloesener/MFRC522_I2C
- This edition: Martin Oswald (@mad), <https://1MHz.de>

License

C64uRemote is released under the **MIT License**. The full text is in the file `LICENSE` in the project root.

It originates from **C64uRemote by Karl Prosser (@klumtsy)**, <https://github.com/ReadyOS-C64/C64uRemote>, which he published under the MIT License. This version is a derivative work and is released under the same terms:

- Copyright (c) 2026 Karl Prosser – original project
- Copyright (c) 2026 Martin Oswald (@mad, <https://1MHz.de>) – port and extensions

The MIT License allows you to use, modify and redistribute the software, including commercially. The only condition: **the copyright notice and the license text must be kept** and included with every copy. There is no warranty and no liability.

The libraries used carry their own licenses: M5Unified and M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) and MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). PlatformIO fetches them at build time.

How this was made

The port, the extensions and these manuals were written with the help of Claude (Anthropic). Concept, idea, hardware decisions and every test on real devices: Martin Oswald (@mad).